

MOD-07: React Optimization & Production Readiness

Enterprise React Specialist Handbook · Principal Frontend Architect track, HCLSoftware

Sep 18, 2026 · @Someone

Chapter scope and standing conventions

This is the terminal module of the core curriculum, and it is deliberately **diagnostic rather than generative**. Every technique here is a response to a measured problem. A learner who applies these tools without a profile in front of them will slow the application down while believing they optimized it — which is the single most common outcome of "performance work" in enterprise React.

ID	Subtopic	What it depends on
MOD-07.1	<code>React.lazy</code> and dynamic imports, chunking, retry on stale hashes	MOD-01.5 ES modules, MOD-05.8 routes, MOD-06.2 boundaries
MOD-07.2	Suspense — fallback mechanics, CLS prevention, nesting, <code>useTransition</code>	MOD-07.1, MOD-02.4 microtask scheduling
MOD-07.3	Profiling, <code>React.memo</code> audits, virtualization, context isolation, bundle budgets, Core Web Vitals, heap analysis	MOD-04.6 Fiber, MOD-03.5 rAF throttling, MOD-06.1 context

Standing conventions carried forward

- Strict TypeScript.** `strict: true`, zero `any`, no silencing casts. Profiler callbacks are typed against React's own `ProfilerOnRenderCallback`; virtualization boundaries carry explicit index types so an off-by-one is a compile error rather than a blank row.
- Jest + React Testing Library.** Chunk-retry wrappers tested against simulated CDN drops; Suspense transitions tested for `aria-busy` and layout stability; windowing math tested as a pure function and at the DOM-node-count level; profiler commit timing asserted.
- WCAG 2.1 AA.** Skeletons carry `aria-busy`; virtualized rows announce their position within the full set; `prefers-reduced-motion` is honoured everywhere a transition or animation appears.

4. **What follows this module.** APP-01 — Enterprise Build Tooling and CI/CD Pipelines, and then the **SDLC and GenAI Governance** chapter, are compiled directly after MOD-07. That ordering is intentional: APP-01 owns the bundler configuration, chunk-hash strategy and CI budget enforcement that this module's techniques assume exist.

The rule that governs the whole module

No optimization ships without a profile that justifies it. Every `React.memo`, every `useMemo`, every virtualization layer, every split point is a trade: more code, more indirection, more ways to be subtly wrong. A reviewer is entitled to ask "what did the Profiler show before and after?", and "it felt slow" is not an answer. This is also why MOD-07.3 is the largest subtopic in the handbook: measurement is the skill, and the fixes are comparatively easy once the measurement is honest.

Source alignment: built against the learner's handwritten notes on the Virtual DOM (11/08/26) and the bundler's role in optimizing JSX and CSS into compact files (10/09/26), both of which describe the mechanisms this module now measures.

Prerequisite gate. MOD-04.6 (Fiber and reconciliation), MOD-03.5 (rAF throttling and layout thrashing), MOD-05.4 and MOD-05.5 (referential stability) and MOD-06.2 (error boundaries) must answer cold. Nothing here is a new mechanism; it is measurement applied to mechanisms already owned.

Part 1 — High-level overview and architectural connections

Three questions define this module. **What does the user download?** (MOD-07.1). **What do they see while waiting?** (MOD-07.2). **How much work happens per frame once it arrives?** (MOD-07.3). Every enterprise performance complaint reduces to one of the three, and the diagnostic skill is knowing which.

The three subtopics

MOD-07.1 — `React.lazy` and dynamic imports. `import()` returns a promise for a module, which the bundler turns into a separate chunk. `React.lazy` wraps that promise so a component can be rendered before its code exists. The architectural content is chunking strategy — route level versus component level — the named-export problem, and **network failure recovery**: a dynamically imported chunk can 404, and in a continuously deployed product it routinely does.

MOD-07.2 — `Suspense`. A boundary that renders a fallback while a child is not ready. The mechanics are simple; the engineering is in **layout shift prevention** (a fallback whose dimensions differ from the content it replaces is a CLS defect), skeletons versus spinners,

coordinating nested boundaries so a page does not reveal itself in four uncoordinated stages, and `useTransition` for keeping the previous UI interactive while the next one prepares.

MOD-07.3 — Performance and profiling. The largest subtopic: reading Profiler flamegraphs, diagnosing commit cascades, auditing `React.memo` prop comparisons, writing windowing math for 100,000+ rows, isolating context re-renders, enforcing bundle budgets, meeting Core Web Vitals SLAs, and finding retained detached DOM in heap snapshots.

The runtime roots: three direct inheritances

Fiber (MOD-04.6) explains why `React.memo` skips work. React's reconciler walks the Fiber tree, and at each node it asks whether that subtree needs re-rendering.

`React.memo` inserts a comparison gate at one node: if the previous and next props are shallow-equal by `Object.is`, React **does not call the component function** and reuses the entire existing subtree — no render, no diff, no commit for anything beneath it. This is why memo's payoff is proportional to subtree size, why it is nearly worthless on a leaf `<span>`, and why it silently does nothing when a parent passes a fresh object or arrow function: one changed reference reopens the whole gate. Memoization is not a speed trick; it is a **pruning instruction to the Fiber walk**, and that framing is only available to someone who holds MOD-04.6.

Error boundaries (MOD-06.2) catch chunk-load failures from `React.lazy`. When a dynamic import rejects — a 404 on a rotated hash, a CDN edge miss, a corporate proxy block, a dropped connection — `React.lazy` **throws that rejection during render**. Nothing else in React can intercept it. Without a boundary the entire tree unmounts and the user gets a white screen, and this is not an edge case: every deploy that rotates chunk hashes puts every open tab one navigation away from it. MOD-06.2 was taught before MOD-07.1 for exactly this reason, and the dependency runs one way — you may not ship code splitting without boundaries.

rAF and throttling (MOD-03.5) power virtualization scroll handlers. A virtualized list recalculates its visible window on every scroll event, which fires at up to 100Hz. Doing that work synchronously — especially if it reads geometry — is the layout-thrashing failure from MOD-03.1 at high frequency, and it is the fastest way to push INP past 500ms. `requestAnimationFrame` coalescing collapses a burst of scroll events into exactly one calculation per frame, aligned to paint. The correct windowing implementation reads `scrollTop` once, computes indices **arithmetically** from known row height, and never measures the DOM inside the handler.

Dependency topology

```

flowchart TD
    A["MOD-07.1<br/>React.lazy and<br/>dynamic imports"] --> B["MOD-07.2<br/>Suspense boundaries"]
    B --> C["MOD-07.3<br/>Profiling, memo,<br/>virtualization, vitals"]
    D["MOD-06.2<br/>Error boundaries"] ==>|"REQUIRED BEFORE"| A
    E["MOD-04.6<br/>Fiber reconciliation"] -.->|"why memo<br/>prunes the walk"| C
    F["MOD-03.5<br/>rAF throttling"] -.->|"scroll<br/>coalescing"| C
    G["MOD-06.1<br/>Context"] -.->|"blast radius<br/>to isolate"| C
    H["MOD-05.4 / 05.5<br/>Referential stability"] -.->|"makes memo<br/>effective"| C
    C --> I["APP-01<br/>Build tooling and<br/>CI budgets"]
    I --> J["SDLC and GenAI<br/>governance"]
  
```

The double arrow is the hard constraint. The dotted edges are the earlier mechanisms this module measures rather than introduces — which is why a three-subtopic module can carry the weight of a seven-subtopic one.

The diagnostic order this module teaches

Performance work follows a fixed sequence, and skipping a step is how teams spend a sprint optimizing something that was never the problem:

1. **Measure in production conditions** — production build, throttled CPU, real device class. A development build is 3–10× slower and profiles nothing real.
2. **Classify the symptom** — slow initial load, slow interaction, or visual instability. These have disjoint causes and disjoint fixes.
3. **Find the actual cost** in the Profiler or a heap snapshot, not by intuition.
4. **Apply the narrowest fix** that addresses it.
5. **Re-measure and record the delta** in the PR description.

What this chapter defers to APP-01

Bundler configuration (Vite, Rollup options, `manualChunks`), CI bundle-budget enforcement, source-map upload, CDN cache-control and chunk-hash strategy, and Lighthouse CI wiring all belong to **APP-01**. This module explains what to ask for and why; APP-01 configures it. Server Components, streaming SSR and data-fetching Suspense are out of syllabus scope; Suspense is covered here as it applies to code splitting and to libraries that already support it.

Part 2 — Production use cases and enterprise failure modes

These are drawn from two environments where performance is not a nicety: a trading console where a dropped frame is a missed price, and a telemetry portal where operators watch a live feed for a full shift.

MOD-07.1 — Dynamic imports

Use case. A console split by route, with the admin bundle (~800 kB) never shipped to read-only auditors.

Failure mode: the stale deployment hash. CI deploys at 14:00 and rotates every chunk hash: `admin.a1b2c3.js` becomes `admin.d4e5f6.js`, and the old file is removed from the CDN. A trader has had the console open since 09:00. Their `index.html` is the 09:00 build and still references `a1b2c3`. At 14:05 they navigate to the admin view, the dynamic import 404s, `React.lazy` throws during render, and — with no boundary — **the whole application white-screens mid-session**. Every user with an open tab is affected simultaneously, which support experiences as “the portal went down at 2pm” for a deploy that succeeded perfectly. Frequency scales with deploy cadence: daily deploys plus long sessions make this a weekly event.

Failure mode: named exports silently breaking the split. `React.lazy` requires a module with a **default export**. A developer lazy-loads a named export, hits a type error, and “fixes” it by converting to a static import. The chunk quietly rejoins the main bundle, the budget regresses, and nobody notices until the bundle report is read three months later.

Failure mode: splitting at the wrong granularity. Splitting a 4 kB button produces a separate network round trip that costs more than the bytes saved. Fifty such splits turn one request into fifty, and on a high-latency corporate VPN the page is slower than before the “optimization”.

MOD-07.2 — Suspense

Use case. A dashboard where the order-book panel loads behind a Suspense boundary.

Failure mode: CLS above 0.25 from a mismatched fallback. The fallback is a 40 px spinner; the resolved panel is 600 px tall. When data arrives the page jumps 560 px. A trader mid-click hits the wrong control because the button moved under the cursor — in a trading context that is a financial incident, not a UX annoyance. CLS of 0.25 is five times the 0.1 threshold and fails the Core Web Vitals SLA outright.

Failure mode: uncoordinated nested boundaries. Six tiles each wrapped in their own boundary resolve at different moments, so the dashboard assembles itself in six visible stages over two seconds, each one reflowing the grid. The fix is coordinating the

boundaries — one boundary around a group that must appear together — not making each fallback prettier.

Failure mode: the fallback flash. A boundary whose content resolves in 80 ms still shows its skeleton for 80 ms, producing a visible flicker on every navigation. Without `useTransition`, every route change blanks the current page to a skeleton even when the new one is nearly instant. With it, the previous UI stays interactive until the next is ready.

MOD-07.3 — Performance and profiling

Use case. A telemetry portal streaming live metrics over WebSocket into a 5,000-cell grid.

Failure mode: the context cascade. Telemetry arrives every 250 ms and is written into a context value. Every push creates a new value object, so **all 5,000 cells re-render four times a second**. The Profiler shows a commit every 250 ms with 5,000 nodes in the flamegraph and a 180 ms commit duration — meaning the main thread is busy roughly 70% of the time, doing nothing a user asked for. Scrolling stutters, typing in the filter box lags, and `React.memo` on the cells does not help because context bypasses props (MOD-06.1).

Failure mode: measuring geometry inside the scroll handler. A virtualized list calls `getBoundingClientRect()` per row inside `onScroll` to decide what is visible. Each read forces a synchronous layout (MOD-03.1) and the handler fires at 100 Hz. INP measures above 500 ms, the list freezes during fast scrolling, and the Performance panel shows a solid wall of purple layout blocks. The fix is arithmetic: with a known row height, `startIndex = floor(scrollTop / rowHeight)` needs no measurement at all.

Failure mode: `React.memo` silently doing nothing. A team wraps every row in `React.memo` and sees no improvement, because the parent passes `onSelect={() => select(row.id)}` and `style={{ height: 36 }}` — a new function and a new object every render. The memo comparison fails on every row every time, so the team has added a comparison cost to every render and gained nothing. Worse, the failure is invisible: there is no warning, and the Profiler shows the rows rendering exactly as before. The audit habit — check what the parent actually passes — is the skill.

Failure mode: retained detached DOM. Dashboard tiles subscribe to a custom event bus on mount and forget to unsubscribe on unmount. Each subscription closes over the tile's DOM node and its telemetry buffer. After an operator navigates between twenty views in a shift, the heap snapshot shows tens of thousands of **detached HTML elements** retained by listener closures, and the tab is holding 2 GB. The browser eventually kills it mid-shift, losing the operator's context. Nothing errors, and the memory graph is the only evidence.

Failure mode: bundle creep with no budget. No CI budget means the main bundle grows 40 kB per sprint, invisibly. Eighteen months later LCP has drifted from 1.8 s to 4.1 s, no single commit caused it, and there is nothing to revert. A moment-locale import pulling 300 kB of unused locales is the classic single contributor, and tree-shaking cannot remove it because the import is dynamic.

Failure mode: optimizing the wrong layer. A team spends a sprint memoizing components to fix a "slow dashboard" that was actually blocked on a 3-second API call. The Profiler would have shown near-zero render time and a long idle gap. This is the failure the diagnostic order in Part 1 exists to prevent, and it is the most expensive one in the list because it consumes a sprint and changes nothing.

Part 3 — Architectural flowcharts and comparison tables

Diagram 1: chunk loading and retry on deployment hash invalidation

flowchart TD

```

    A["Route navigation<br/>triggers import()"] --> B{"Chunk fetched?"}
    B -- "200 OK" --> C["Module resolves<br/>Suspense reveals content"]
    B -- "404 / network error" --> D["withChunkRetry intercepts"]
    D --> E{"Attempts left?"}
    E -- "Yes" --> F["Wait 500ms x 2^n<br/>+ jitter"]
    F --> G["Re-attempt import()"]
    G --> B
    E -- "No" --> H{"Already reloaded<br/>this session?<br/>(sessionStorage guard)"}
    H -- "No" --> I["Set guard flag<br/>location.reload(true)"]
    I --> J["Fresh index.html<br/>fresh chunk hashes"]
    J --> C
    H -- "Yes" --> K["THROW to route<br/>error boundary"]
    K --> L["'A new version is available'<br/>+ Reload button"]
  
```

Four design decisions carry this, and dropping any one reproduces a Part 2 failure.

1. **Retry first, reload second.** A transient network blip is far more common than a stale deploy, and a retry is invisible to the user while a reload is not.
2. **Exponential backoff with jitter.** Without jitter, every tab in the tenant retries in lockstep and hammers the CDN at the same instants.
3. A **sessionStorage reload guard is mandatory.** Without it, a genuinely missing chunk — a broken deploy — produces an **infinite reload loop**: load, fail, reload, fail. The guard converts the second failure into an honest error page.

4. **The boundary is the terminal state.** When retry and reload are both exhausted, `React.lazy` throws and MOD-06.2's route boundary renders the version-mismatch fallback. This is the concrete dependency that made MOD-06.2 a prerequisite.

Diagram 2: the virtualization windowing engine

```

flowchart TD
    A["scroll event<br/>(up to 100Hz)"] --> B{"rAF already<br/>queued?"}
    B -- "Yes" --> C["Drop this event:<br/>one calc per frame"]
    B -- "No" --> D["requestAnimationFrame"]
    D --> E["Read scrollTop ONCE<br/>(single geometry read)"]
    E --> F["startIndex =<br/>floor(scrollTop / rowHeight)"]
    F --> G["visibleCount =<br/>ceil(viewportH / rowHeight)"]
    G --> H["start = max(0, startIndex - overscan)<br/>end = min(total,<br/>startIndex + visibleCount + overscan)"]
    H --> I{"Indices changed?"}
    I -- "No" --> J["Bail out:<br/>no setState, no render"]
    I -- "Yes" --> K["setState({start, end})"]
    K --> L["Spacer height =<br/>total x rowHeight"]
    L --> M["translate3d(0, start x rowHeight, 0)<br/>on the row container"]
    M --> N["Render rows[start..end]<br/>CONSTANT node count"]
  
```

The three properties that make this fast. No DOM measurement anywhere — with a known row height, every index is arithmetic. The `translate3d` transform is **composed**, not laid out, so scrolling moves pixels on the GPU rather than triggering reflow (MOD-03.1's transform-over-top rule). And the bail-out when indices are unchanged means a 3-pixel scroll costs one subtraction, not a React commit.

Overscan is the buffer of rows rendered outside the viewport, typically 3–5. Too little and fast scrolling shows blank gaps before the next frame paints; too much and you are rendering rows nobody sees. It is the one tunable in the engine, and it should be measured rather than guessed.

Table 1: code-splitting decision matrix

	Route level	Heavy modal / drawer	Below-the-fold widget	Library / vendor chunk
Typical saving	100–800 kB — the largest single win available	50–300 kB (charts, editors, PDF)	10–80 kB	100–400 kB across routes

	Route level	Heavy modal / drawer	Below-the-fold widget	Library / vendor chunk
Loads when	The route is navigated to	The control is opened	It scrolls into view	First route that needs it
Perceived latency	Covered by a route transition	Visible — user just clicked	Invisible if prefetched	Hidden behind the initial load
CLS risk	Low — whole page replaced	High — inserted into a live layout	High unless space reserved	None
Fallback needed	Route-level skeleton	Sized skeleton matching final dimensions	Reserved-height placeholder	None
Retry wrapper required	Yes — long-lived tabs hit stale hashes	Yes	Yes	Yes
Prefetch strategy	On link hover / focus	On hover of the trigger control	On approach via IntersectionObserver	With its first consumer
Verdict	Always split. The default unit of code splitting	Split anything over ~50 kB used by a minority	Split if genuinely below the fold	Split shared vendor code; configured in APP-01

Do not split anything under roughly 30 kB, anything on the critical render path, or anything needed immediately after mount — the round trip costs more than the bytes.

Table 2: performance profiling diagnostic matrix

Observed symptom	Profiler / DevTools pattern	Root cause	Architectural fix
Typing lags across the page	One commit per keystroke containing hundreds of unrelated components	Context value identity changing, or state lifted too high	Split contexts by volatility; colocate the input's state (MOD-06.1)

Observed symptom	Profiler / DevTools pattern	Root cause	Architectural fix
A list re-renders entirely on one row's change	Every row bar is coloured in the flamegraph	Missing or defeated <code>React.memo</code> ; <code>unstable_onSelect</code> or <code>style</code> props	<code>React.memo</code> plus stable handlers via <code>useCallback</code> (MOD-05.5)
Slow interaction, near-zero render time	Long idle gap between interaction and commit	The bottleneck is the network, not React	Fix the request; optimistic UI; a server-state library (MOD-06.6)
Scroll stutters, INP > 500 ms	Solid purple layout blocks in the Performance panel	Forced synchronous layout inside the scroll handler	rAF coalescing plus arithmetic windowing (MOD-03.5)
Dashboard slows over a shift	Heap grows monotonically; detached elements in the snapshot	Listeners, timers or subscriptions never torn down	Cleanup in every effect; <code>{ signal }</code> for listeners (MOD-05.2)
One tile freezes the page every few seconds	A single very wide bar in the flamegraph	An expensive derivation running on every render	<code>useMemo</code> with measured cost, or move the work off the main thread
High initial load, fast thereafter	Huge first commit; one enormous bundle in the network panel	No code splitting	Route-level <code>React.lazy</code> (MOD-07.1)
Page jumps as content arrives	CLS spikes in the Performance panel	Fallbacks not matching final dimensions	Sized skeletons; reserve space (MOD-07.2)
A memoized component still re-renders	Profiler says "props changed" and names the prop	An inline object, array or arrow function from the parent	Stabilize at the source, not by adding more memos

How to read "why did this render?" React DevTools Profiler, with "Record why each component rendered" enabled, names the changed prop or the hook index. That attribution is the entire diagnostic step — everything above is a lookup once you have it.

Table 3: Core Web Vitals SLA matrix

Metric	Enterprise target	What it measures	Typical React SPA culprit	Remediation
LCP	< 2.5 s (good), < 4.0 s (needs work)	Time until the largest content element paints	Monolithic bundle; blocking data fetch before first paint; unoptimized hero images	Route-level splitting; server-render or prerender the shell; preload critical assets; compress images and set explicit dimensions
INP	< 200 ms (good), < 500 ms (needs work)	Latency from interaction to next paint, across the whole session	Long tasks: unthrottled scroll handlers, context cascades, large synchronous renders, DOM measurement in handlers	rAF throttling; virtualization; <code>useTransition</code> for non-urgent updates; break long tasks with <code>scheduler.yield</code> where available
CLS	< 0.1 (good), < 0.25 (needs work)	Cumulative unexpected layout shift	Suspense fallbacks sized differently from content; images without dimensions; late-inserted banners	Sized skeletons; explicit <code>width / height</code> on media; reserve space for asynchronous content; never insert above existing content

INP replaced FID in March 2024 and is a materially harder target: FID measured only the delay before a handler started, while INP measures the full path to the next paint. Applications that comfortably passed FID routinely fail INP, and virtualization plus context isolation are the two highest-leverage fixes in a data-dense React portal.

Measure in the field, not only in the lab. Lighthouse is a single cold load on synthetic hardware. Real User Monitoring via the `web-vitals` library, segmented by device class and region, is what reveals that the portal is fine in the London office and unusable on a mid-range laptop over VPN from Bengaluru. The SLA is met at the 75th percentile of real users, not on a developer's machine.

Part 4 — Low-level deep dive and interactive coding lab

Strict TypeScript throughout. Work each debug challenge before reading its diagnosis.

MOD-07.1 — `React.lazy` and dynamic imports

Mechanism. `import('./Module')` returns a `Promise<Module>` and signals the bundler to emit a separate chunk. `React.lazy(factory)` takes a function returning that promise and produces a component that, on first render, throws the promise — which the nearest `Suspense` boundary catches and converts into a fallback. On resolution React re-renders with the real component. If the promise **rejects**, `React.lazy` throws the error instead, and only an error boundary can catch it.

```
// Bad – named export mishandled, no retry, no boundary, split too fine
const AdminPanel = lazy(() => import('./AdminPanel')); // AdminPanel is a
NAMED export
const SaveButton = lazy(() => import('./SaveButton')); // 4 kB: a pointless
round trip

<Suspense fallback={<Spinner />}>
  <AdminPanel /> /* a chunk 404 white-screens the app */
</Suspense>

// Production – typed named-export adapter, retry wrapper, boundary above
import { lazy, Suspense, type ComponentType } from 'react';
import { withChunkRetry } from './withChunkRetry';

// Adapts a named export to the default shape React.lazy requires, with types
intact
function lazyNamed<TModule, TKey extends keyof TModule>(
  factory: () => Promise<TModule>,
  name: TKey,
): React.LazyExoticComponent<TModule[TKey] & ComponentType<never>> {
  return lazy(() =>
    withChunkRetry(factory).then(module => ({
      default: module[name] as TModule[TKey] & ComponentType<never>,
    })),
  );
}

const AdminPanel = lazyNamed(() => import('./AdminPanel'), 'AdminPanel');

<ErrorBoundary tier="route" fallback={s => <RouteFallback {...s} />}>
  <Suspense fallback={<AdminPanelSkeleton />}>
    <AdminPanel />
  </Suspense>
</ErrorBoundary>
```

Boundary ordering matters and is frequently reversed. The error boundary goes **outside** the Suspense boundary. Inside, a chunk-load rejection unmounts the Suspense boundary along with its fallback and the error escapes upward anyway — so the fallback you wrote never renders.

Prefetching on intent removes the perceived latency without shipping the bytes up front:

```
const prefetchAdmin = (): void => { void import('./AdminPanel'); };

<Link to="/admin" onMouseEnter={prefetchAdmin} onFocus={prefetchAdmin}>
  Admin
</Link>
```

`onFocus` alongside `onMouseEnter` is not optional — keyboard users reach the link by focus, and prefetching only on hover gives them a slower application than mouse users (an SC 2.1.1-adjacent equity issue even though it is not a conformance failure).

Debug challenge DBG-07.1

```
function Dashboard(): React.ReactElement {
  const [showChart, setShowChart] = useState(false);
  const Chart = lazy(() => import('./HeavyChart'));

  return (
    <>
      <button type="button" onClick={() => setShowChart(true)}>Show
chart</button>
      {showChart && (
        <Suspense fallback={<div>Loading...</div>}>
          <Chart />
        </Suspense>
      )}
    </>
  );
}
```

► Reveal diagnosis

`lazy()` is called **inside the component body**, so a new lazy component is created on every render. Each one has its own internal state, so React unmounts the previous chart and remounts a fresh one on every parent render — the chart's own state is destroyed, its data refetches, and the fallback flashes repeatedly. It is the MOD-04.6 type-identity rule: a new component type at the same position means destroy and rebuild. Hoist `const Chart = lazy(() => import('./HeavyChart'))` to module scope. The module cache means the network request still happens only once; it is the *component identity* that must

be stable. Two further defects: there is no error boundary above the Suspense boundary, and the `<div>Loading...</div>` fallback is unsized, so revealing a 500 px chart produces a CLS spike.

MOD-07.2 — Suspense

Mechanism. A Suspense boundary catches a thrown promise from anywhere in its subtree, renders `fallback` until that promise settles, then renders the children. The **nearest** boundary wins, which is what makes nesting a coordination tool: place one boundary around a group that must appear together, and separate boundaries around parts that may appear independently. `useTransition` marks a state update as non-urgent, so React keeps showing the current UI — interactive — instead of replacing it with a fallback.

```
// Bad – spinner of the wrong size, one boundary per tile, no transition
<Suspense fallback={<Spinner />}>           {/* 40px tall; content is 600px
*/}
  <OrderBook />
</Suspense>

// Production – dimension-matched skeleton, coordinated grouping, accessible
interface SkeletonProps {
  readonly height: number;
  readonly label: string;
}

function PanelSkeleton({ height, label }: SkeletonProps): React.ReactElement
{
  return (
    <div
      role="status"
      aria-busy="true"
      aria-label={label}
      // EXACTLY the height of the resolved panel: zero layout shift
      style={{ height, contain: 'strict' }}
      className="skeleton" // animation gated on prefers-reduced-motion in
CSS
    >
      <span className="sr-only">{label}</span>
    </div>
  );
}

// One boundary around the group that must appear together
<Suspense fallback={<PanelSkeleton height={600} label="Loading trading
```



```

panels" />}>
  <OrderBook />
  <DepthChart />
</Suspense>

// A separate boundary for independent, lower-priority content
<Suspense fallback={<PanelSkeleton height={240} label="Loading activity feed"
/>}>
  <ActivityFeed />
</Suspense>

```

```

/* Reduced motion is a conformance requirement, not a preference */
@media (prefers-reduced-motion: reduce) {
  .skeleton { animation: none; }
}

```

useTransition for navigation without a fallback flash:

```

const [isPending, startTransition] = useTransition();

const goToTenant = (id: string): void => {
  startTransition(() => {
    navigate(`/tenants/${id}`); // previous page stays visible and
    interactive
  });
};

<button type="button" onClick={() => goToTenant(id)} aria-busy={isPending}>
  {isPending ? 'Opening...' : 'Open tenant'}
</button>

```

aria-busy on the trigger is what tells a screen-reader user that the pending state exists at all — visually it is a label change, and without the attribute the transition is silent.

Debug challenge DBG-07.2

```

<Suspense fallback={<Skeleton />}>
  <TenantList tenantId={tenantId} />
</Suspense>

```

Switching tenants blanks the whole list to the skeleton for ~800 ms each time, and users report that the app "resets" on every switch.

► **Reveal diagnosis**

The tenant change is treated as an **urgent** update, so React discards the current list immediately and shows the fallback while the new data loads. Wrap the state change in `startTransition` and React keeps the previous list mounted and interactive until the new one is ready — no blanking, no scroll-position loss. Pair it with `isPending` to dim the list and set `aria-busy` so the state is perceivable. Note the boundary between the two tools: Suspense fallbacks are correct for a **first** load where there is nothing to keep, and transitions are correct for a **subsequent** load where there is. Using a fallback for both is what produces the "the app resets" complaint.

MOD-07.3 — Performance and profiling

Mechanism. The `<Profiler>` component reports a callback on every commit within its subtree, with the phase (`mount` or `update`), `actualDuration` (time spent rendering this commit) and `baseDuration` (estimated time without any memoization). `React.memo` inserts a shallow prop comparison at a Fiber node; when it passes, React skips the component function and reuses the whole subtree.

```
// Bad – memo defeated by two inline props, expensive derivation per render
const Row = memo(function Row({ item, onSelect, style }: RowProps) {
  return <div style={style} onClick={onSelect}>{item.name}</div>;
});

{rows.map(row => (
  <Row
    key={row.id}
    item={row}
    onSelect={() => select(row.id)} // new function: memo fails
    style={{ height: 36 }}         // new object: memo fails
  />
))}

// Production – stable handler by id, hoisted style, typed profiler
const ROW_STYLE: React.CSSProperties = { height: 36 }; // module scope: one object

interface RowProps {
  readonly item: TelemetryRow;
  readonly onSelect: (id: string) => void; // takes the id, so it can be stable
}

const Row = memo(function Row({ item, onSelect }: RowProps):
React.ReactElement {
  return (
```

```

    <div style={ROW_STYLE} onClick={() => onSelect(item.id)}>
      {item.name}
    </div>
  );
});

// In the parent: one stable handler for every row
const handleSelect = useCallback((id: string): void => {
  setSelected(current => {
    const next = new Set(current);
    if (next.has(id)) next.delete(id);
    else next.add(id);
    return next;
  });
}, []);

// Typed profiler callback — React exports the exact signature
const onRender: React.ProfilerOnRenderCallback = (
  id,
  phase,
  actualDuration,
  baseDuration,
  startTime,
  commitTime,
) => {
  if (actualDuration > 16) {
    telemetry.track('slow_commit', { id, phase, actualDuration, baseDuration,
    commitTime });
  }
};

<Profiler id="telemetry-feed" onRender={onRender}>
  <VirtualizedTelemetryFeed rows={rows} onSelect={handleSelect} />
</Profiler>

```

The inline arrow **inside** `Row` is fine: it is a new function per row render, but `Row` only renders when its own props change, and the arrow is not itself a memo input. The distinction — stable props at the boundary, free allocation inside — is the whole memo audit in one sentence.

Context isolation for a live feed. The telemetry cascade from Part 2 is fixed by keeping fast-changing data out of context entirely: hold it in a store with per-subscriber selection (MOD-06.4), or split the context so cells subscribe only to their own slice. `React.memo` cannot save a context consumer.

Heap profiling for detached DOM. Take a snapshot, interact, navigate away, force garbage collection, take a second snapshot, and filter for **Detached** nodes. A retained detached element is always held by something — a listener, a timer, a closure in a subscription, or a stale ref. The retainer path in the snapshot names it directly.

Debug challenge DBG-07.3

```
const TelemetryGrid = memo(function TelemetryGrid({
  rows,
  thresholds,
}): {
  readonly rows: readonly TelemetryRow[];
  readonly thresholds: Thresholds;
}) {
  const breaches = rows.filter(r => r.value > thresholds.max);
  return <Grid rows={rows} breaches={breaches} />;
});

// Parent, re-rendering every 250ms from a WebSocket push
<TelemetryGrid rows={rows} thresholds={{ max: 100, min: 0 }} />
```

► Reveal diagnosis

The memo never engages. `thresholds` is a fresh object literal on every parent render, so the shallow comparison fails four times a second and `TelemetryGrid` re-renders regardless — along with the `filter` over every row. The Profiler's "why did this render?" would name `thresholds` directly. Two fixes, both required: hoist the literal to module scope (it is constant) or memoize it at the source with `useMemo`; and memoize the derivation with `useMemo(() => rows.filter(...), [rows, thresholds])` so it does not re-run when only an unrelated prop changes. Note what *not* to do: adding a custom `areEqual` comparator that deep-compares `thresholds` moves the cost into the comparison and runs it on every render — fixing the symptom at the wrong layer. **Stabilize at the source; never compensate downstream.**

Coding Lab LAB-07 — `VirtualizedTelemetryFeed` and resilient code splitting

Brief. Two production modules and the feed that composes them: a dynamic-import wrapper that survives CDN drops and deployment hash rotation, and a zero-dependency accessible virtualized window that renders 100,000 telemetry records at a constant DOM cost.

1. **withChunkRetry.ts**

```
// withChunkRetry.ts

const RELOAD_GUARD_KEY = 'chunk-reload-attempted';

export interface ChunkRetryOptions {
  readonly retries?: number;
  readonly baseDelayMs?: number;
  readonly maxDelayMs?: number;
  /** Injected for tests; defaults to the real reload. */
  readonly reload?: () => void;
  readonly storage?: Pick<Storage, 'getItem' | 'setItem' | 'removeItem'>;
}

/** A chunk failure looks different across bundlers and browsers. */
export function isChunkLoadError(error: unknown): boolean {
  if (!(error instanceof Error)) return false;
  return (
    /loading chunk \d+ failed/i.test(error.message) ||
    /failed to fetch dynamically imported module/i.test(error.message) ||
    /importing a module script failed/i.test(error.message) ||
    error.name === 'ChunkLoadError'
  );
}

const sleep = (ms: number): Promise<void> =>
  new Promise(resolve => window.setTimeout(resolve, ms));

/**
 * Wraps a dynamic import with exponential retry and a one-shot cache-busting
 * reload for stale deployment hashes. Never loops: the sessionStorage guard
 * converts a second failure into a throw the route error boundary handles.
 */
export async function withChunkRetry<T>(
  factory: () => Promise<T>,
  options: ChunkRetryOptions = {},
): Promise<T> {
  const {
    retries = 3,
    baseDelayMs = 500,
    maxDelayMs = 4_000,
    reload = () => window.location.reload(),
    storage = window.sessionStorage,
  } = options;

```

```

for (let attempt = 0; attempt <= retries; attempt++) {
  try {
    const module = await factory();
    storage.removeItem(RELOAD_GUARD_KEY); // success clears the guard
    return module;
  } catch (error) {
    const isLast = attempt === retries;

    // A genuine module error (a throw inside the module) must not be
    retried
    if (!isChunkLoadError(error)) throw error;

    if (!isLast) {
      const delay = Math.min(baseDelayMs * 2 ** attempt, maxDelayMs);
      const jitter = Math.random() * delay * 0.3; // avoid a lockstep CDN
      stampede
      await sleep(delay + jitter);
      continue;
    }

    // Retries exhausted: this is probably a rotated deployment hash.
    if (storage.getItem(RELOAD_GUARD_KEY) === null) {
      storage.setItem(RELOAD_GUARD_KEY, String(Date.now()));
      reload(); // fresh index.html, fresh hashes
      // Never resolves: the page is navigating away
      return await new Promise<T>(() => {});
    }

    // Already reloaded once this session: stop, and let the boundary
    render
    throw error;
  }
  throw new Error('withChunkRetry: unreachable');
}

```

The `isChunkLoadError` guard is what keeps this honest: retrying a real error thrown *inside* the module would hide a genuine bug behind three silent retries and a page reload.

2. VirtualizedWindow.tsx

```

// VirtualizedWindow.tsx
import { useCallback, useEffect, useId, useRef, useState } from 'react';

// ----- Pure windowing math: unit-testable with no DOM -----

```

```

export interface WindowRange {
  readonly startIndex: number;
  readonly endIndex: number;    // exclusive
  readonly offsetY: number;
}

export interface WindowInput {
  readonly scrollTop: number;
  readonly viewportHeight: number;
  readonly rowHeight: number;
  readonly totalRows: number;
  readonly overscan: number;
}

export function computeWindow({
  scrollTop,
  viewportHeight,
  rowHeight,
  totalRows,
  overscan,
}: WindowInput): WindowRange {
  if (rowHeight <= 0 || totalRows === 0) {
    return { startIndex: 0, endIndex: 0, offsetY: 0 };
  }
  const clampedScroll = Math.max(0, Math.min(scrollTop, totalRows *
rowHeight));
  const firstVisible = Math.floor(clampedScroll / rowHeight);
  const visibleCount = Math.ceil(viewportHeight / rowHeight);

  const startIndex = Math.max(0, firstVisible - overscan);
  const endIndex = Math.min(totalRows, firstVisible + visibleCount +
overscan);

  return { startIndex, endIndex, offsetY: startIndex * rowHeight };
}

// ----- Component -----

export interface VirtualizedWindowProps<T> {
  readonly items: readonly T[];
  readonly rowHeight: number;
  readonly height: number;
  readonly overscan?: number;
  readonly label: string;
  readonly getKey: (item: T, index: number) => string;
  readonly renderRow: (item: T, index: number) => React.ReactNode;
}

```



```

}

export function VirtualizedWindow<T>({
  items,
  rowHeight,
  height,
  overscan = 4,
  label,
  getKey,
  renderRow,
}: VirtualizedWindowProps<T>): React.ReactElement {
  const viewportRef = useRef<HTMLDivElement>(null);
  const frameRef = useRef<number | null>(null);
  const statusId = useId();
  const [activeIndex, setActiveIndex] = useState(0);

  const [range, setRange] = useState<WindowRange>(() =>
    computeWindow({
      scrollTop: 0,
      viewportHeight: height,
      rowHeight,
      totalRows: items.length,
      overscan,
    }),
  );

  // rAF-coalesced: at most ONE calculation per frame (MOD-03.5)
  const handleScroll = useCallback(()> void => {
    if (frameRef.current !== null) return; // a frame is already
    queued
    frameRef.current = window.requestAnimationFrame(() => {
      frameRef.current = null;
      const el = viewportRef.current;
      if (el === null) return;
      // ONE geometry read, no per-row measurement (MOD-03.1)
      const next = computeWindow({
        scrollTop: el.scrollTop,
        viewportHeight: height,
        rowHeight,
        totalRows: items.length,
        overscan,
      });
      setRange(current =>
        current.startIndex === next.startIndex && current.endIndex ===
next.endIndex
        ? current // bail out: no commit

```

```

        : next,
      );
    });
  }, [height, rowHeight, items.length, overscan]);

  useEffect(() => {
    return () => {
      if (frameRef.current !== null) {
        window.cancelAnimationFrame(frameRef.current); // no leaked frame
        frameRef.current = null;
      }
    };
  }, []);

  const scrollToIndex = useCallback(
    (index: number): void => {
      const clamped = Math.max(0, Math.min(index, items.length - 1));
      setActiveIndex(clamped);
      viewportRef.current?.scrollTo({ top: clamped * rowHeight, behavior:
'auto' });
    },
    [items.length, rowHeight],
  );

  const handleKeyDown = useCallback(
    (event: React.KeyboardEvent<HTMLDivElement>): void => {
      const pageSize = Math.floor(height / rowHeight);
      const moves: Record<string, number | undefined> = {
        ArrowDown: activeIndex + 1,
        ArrowUp: activeIndex - 1,
        PageDown: activeIndex + pageSize,
        PageUp: activeIndex - pageSize,
        Home: 0,
        End: items.length - 1,
      };
      const next = moves[event.key];
      if (next === undefined) return;
      event.preventDefault();
      scrollToIndex(next);
    },
    [activeIndex, height, rowHeight, items.length, scrollToIndex],
  );

  const visible = items.slice(range.startIndex, range.endIndex);

  return (

```

```

<>
<div
  ref={viewportRef}
  onScroll={handleScroll}
  onKeyDown={handleKeyDown}
  tabIndex={0}
  role="grid"
  aria-label={label}
  aria-rowcount={items.length}          // the TRUE total, not the slice
  aria-describedby={statusId}
  style={{ height, overflowY: 'auto', position: 'relative' }}
>
  {/* Spacer establishes the full scrollable height */}
  <div style={{ height: items.length * rowHeight, position: 'relative'
}}>

    {/* Composited transform: no reflow while scrolling */}
    <div
      style={{
        transform: `translate3d(0, ${range.offsetY}px, 0)`,
        position: 'absolute',
        top: 0,
        left: 0,
        right: 0,
        willChange: 'transform',
      }}
    >
      {visible.map((item, i) => {
        const absoluteIndex = range.startIndex + i;
        return (
          <div
            key={getKey(item, absoluteIndex)}
            role="row"
            aria-rowindex={absoluteIndex + 1}    // 1-based, absolute
            aria-selected={absoluteIndex === activeIndex}
            style={{ height: rowHeight }}
          >
            {renderRow(item, absoluteIndex)}
          </div>
        );
      })}
    </div>
  </div>

  {/* Politely announces position within the full set, not the rendered
slice */}

```

```

        <p id={statusId} role="status" aria-live="polite" className="sr-only">
          Showing rows {range.startIndex + 1} to {range.endIndex} of
        {items.length}
        </p>
      </>
    );
  }
}

```

Why the accessibility details are not optional. `aria-rowcount` reports 100,000 — a virtualized grid that reports only its 40 rendered rows tells a screen-reader user the dataset has 40 rows, which is worse than no ARIA at all. `aria-rowindex` is absolute, so "row 48,201 of 100,000" is meaningful. The live region announces the window position politely, so a keyboard user paging through knows where they are. And `willChange: 'transform'` promotes the row container to its own compositing layer, which is what keeps the scroll on the GPU.

3. `VirtualizedTelemetryFeed.tsx` — composing it all

```

import { memo, useCallback, useMemo, Profiler, Suspense } from 'react';
import { VirtualizedWindow } from './VirtualizedWindow';

export interface TelemetryRow {
  readonly id: string;
  readonly agent: string;
  readonly metric: string;
  readonly value: number;
  readonly at: number;
}

const ROW_STYLE: React.CSSProperties = { display: 'flex', alignItems: 'center' };

interface FeedRowProps {
  readonly row: TelemetryRow;
  readonly breached: boolean;
  readonly onSelect: (id: string) => void;
}

// memo works here because every prop is stable or primitive
const FeedRow = memo(function FeedRow({
  row,
  breached,
  onSelect,
}: FeedRowProps): React.ReactElement {
  return (

```

```

    <div style={ROW_STYLE} onClick={() => onSelect(row.id)}>
      <span>{row.agent}</span>
      <span>{row.metric}</span>
      <span>{row.value}</span>
      {/* Text, not colour alone (SC 1.4.1) */}
      {breached && <span> – over threshold</span>}
    </div>
  );
});

export interface VirtualizedTelemetryFeedProps {
  readonly rows: readonly TelemetryRow[];
  readonly threshold: number;      // primitive: memo-friendly
  readonly onSelect: (id: string) => void;
}

export function VirtualizedTelemetryFeed({
  rows,
  threshold,
  onSelect,
}: VirtualizedTelemetryFeedProps): React.ReactElement {
  // Derived once per data change, not per render
  const breachedIds = useMemo(
    () => new Set(rows.filter(r => r.value > threshold).map(r => r.id)),
    [rows, threshold],
  );

  const getKey = useCallback((row: TelemetryRow): string => row.id, []);

  const renderRow = useCallback(
    (row: TelemetryRow): React.ReactNode => (
      <FeedRow row={row} breached={breachedIds.has(row.id)} onSelect=
{onSelect} />
    ),
    [breachedIds, onSelect],
  );

  return (
    <VirtualizedWindow<TelemetryRow>
      items={rows}
      rowHeight={36}
      height={720}
      overscan={4}
      label="Live telemetry feed"
      getKey={getKey}

```

```

        renderRow={renderRow}
      />
    );
  }

```

4. Jest + React Testing Library suite

```

import { render, screen, waitFor, act } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import { withChunkRetry, isChunkLoadError } from './withChunkRetry';
import { computeWindow, VirtualizedWindow } from './VirtualizedWindow';
import { VirtualizedTelemetryFeed, type TelemetryRow } from
'./VirtualizedTelemetryFeed';

// ===== 1. withChunkRetry =====

const chunkError = (): Error =>
  Object.assign(new Error('Failed to fetch dynamically imported module'), {
    name: 'ChunkLoadError',
  });

const memoryStorage = () => {
  const map = new Map<string, string>();
  return {
    getItem: (k: string) => map.get(k) ?? null,
    setItem: (k: string, v: string) => { map.set(k, v); },
    removeItem: (k: string) => { map.delete(k); },
  };
};

describe('withChunkRetry', () => {
  beforeEach(() => jest.useFakeTimers());
  afterEach(() => jest.useRealTimers());

  it('resolves on the first attempt without scheduling a retry', async () =>
  {
    const factory = jest.fn().mockResolvedValue({ default: 'Module' });
    await expect(withChunkRetry(factory, { storage: memoryStorage()
  })).resolves.toEqual({
      default: 'Module',
    });
    expect(factory).toHaveBeenCalledTimes(1);
  });

  it('RECOVERS from a simulated CDN drop on the third attempt', async () => {
    const factory = jest

```

```

        .fn()
        .mockRejectedValueOnce(chunkError())
        .mockRejectedValueOnce(chunkError())
        .mockResolvedValue({ default: 'Module' });

    const promise = withChunkRetry(factory, { storage: memoryStorage(),
baseDelayMs: 100 });
    await jest.runAllTimersAsync();
    await expect(promise).resolves.toBeDefined();
    expect(factory).toHaveBeenCalledTimes(3);
  });

it('grows the delay exponentially between attempts', async () => {
  const spy = jest.spyOn(window, 'setTimeout');
  const factory = jest.fn().mockRejectedValue(chunkError());
  const reload = jest.fn();
  const promise = withChunkRetry(factory, {
    storage: memoryStorage(),
    baseDelayMs: 100,
    retries: 3,
    reload,
  }).catch(() => {});
  await jest.runAllTimersAsync();
  await promise;
  const delays = spy.mock.calls.map(([, ms]) => Number(ms));
  expect(delays[1]).toBeGreaterThan(delays[0]);
  expect(delays[2]).toBeGreaterThan(delays[1]);
});

it('RELOADS once when retries are exhausted (stale deployment hash)', async
() => {
  const factory = jest.fn().mockRejectedValue(chunkError());
  const reload = jest.fn();
  const storage = memoryStorage();
  void withChunkRetry(factory, { storage, baseDelayMs: 10, retries: 2,
reload });
  await jest.runAllTimersAsync();
  expect(reload).toHaveBeenCalledTimes(1);
  expect(storage.getItem('chunk-reload-attempted')).not.toBeNull();
});

it('does NOT reload twice: throws to the boundary on the second failure',
async () => {
  const factory = jest.fn().mockRejectedValue(chunkError());
  const reload = jest.fn();
  const storage = memoryStorage();

```

```

    storage.setItem('chunk-reload-attempted', '1'); // a reload already
    happened

    const promise = withChunkRetry(factory, { storage, baseDelayMs: 10,
retries: 1, reload });
    await jest.runAllTimersAsync();
    await expect(promise).rejects.toThrow();
    expect(reload).not.toHaveBeenCalled(); // no infinite reload
loop
  });

  it('does NOT retry a genuine error thrown inside the module', async () => {
    const factory = jest.fn().mockRejectedValue(new TypeError('x is not a
function'));
    await expect(
      withChunkRetry(factory, { storage: memoryStorage() }),
    ).rejects.toBeInstanceOf(TypeError);
    expect(factory).toHaveBeenCalledTimes(1);
  });

  it('clears the reload guard after a successful load', async () => {
    const storage = memoryStorage();
    storage.setItem('chunk-reload-attempted', '1');
    await withChunkRetry(jest.fn().mockResolvedValue({}), { storage });
    expect(storage.getItem('chunk-reload-attempted')).toBeNull();
  });

  it('recognises chunk errors across bundler dialects', () => {
    expect(isChunkLoadError(new Error('Loading chunk 42
failed'))).toBe(true);
    expect(isChunkLoadError(new Error('Importing a module script
failed'))).toBe(true);
    expect(isChunkLoadError(new Error('Cannot read properties of
null'))).toBe(false);
  });
});

// ===== 2. Suspense fallback behaviour =====

describe('Suspense fallback', () => {
  it('announces loading with aria-busy and reserves the final height', async
() => {
    let resolve!: (v: { default: React.ComponentType }) => void;
    const Lazy = React.lazy(() => new Promise<{ default: React.ComponentType
}>(r => { resolve = r; }));

```



```

render(
  <Suspense
    fallback={
      <div role="status" aria-busy="true" aria-label="Loading panel"
style={{ height: 600 }} />
    }
  >
    <Lazy />
  </Suspense>,
);

const skeleton = screen.getByRole('status', { name: 'Loading panel' });
expect(skeleton).toHaveAttribute('aria-busy', 'true');
expect(skeleton).toHaveStyle({ height: '600px' }); // same height as
content

act(() => resolve({ default: () => <div style={{ height: 600
}}>Panel</div> }));
await waitFor(() =>
expect(screen.getByText('Panel')).toBeInTheDocument());

// Fallback gone, and it never had a different height: no layout flash
expect(screen.queryByRole('status')).not.toBeInTheDocument();
});
});

// ===== 3. Windowing math (pure) =====

describe('computeWindow', () => {
  const base = { viewportHeight: 720, rowHeight: 36, totalRows: 100_000,
overscan: 4 };

  it('returns the first slice at scrollTop 0, clamped at the start', () => {
    const r = computeWindow({ ...base, scrollTop: 0 });
    expect(r.startIndex).toBe(0); // clamped, not -4
    expect(r.endIndex).toBe(24); // 20 visible + 4 overscan
    expect(r.offsetY).toBe(0);
  });

  it('computes indices arithmetically for a deep scroll', () => {
    const r = computeWindow({ ...base, scrollTop: 50_000 });
    expect(r.startIndex).toBe(Math.floor(50_000 / 36) - 4);
    expect(r.offsetY).toBe(r.startIndex * 36);
  });

  it('clamps endIndex at the last row', () => {

```

```

    const r = computeWindow({ ...base, scrollTop: 100_000 * 36 });
    expect(r.endIndex).toBe(100_000);
  });

  it('handles an empty list without throwing', () => {
    expect(computeWindow({ ...base, scrollTop: 0, totalRows: 0 })).toEqual({
      startIndex: 0,
      endIndex: 0,
      offsetY: 0,
    });
  });
});

// ===== 4. VirtualizedWindow (DOM) =====

const rows: readonly TelemetryRow[] = Array.from({ length: 100_000 }, (_, i)
=> ({
  id: `r-${i}`,
  agent: `agent-${i % 50}`,
  metric: 'cpu',
  value: i % 200,
  at: i,
})));

const renderFeed = () =>
  render(<VirtualizedTelemetryFeed rows={rows} threshold={150} onSelect=
{jest.fn()} />);

const scrollTo = async (px: number): Promise<void> => {
  const viewport = screen.getByRole('grid');
  Object.defineProperty(viewport, 'scrollTop', { value: px, configurable:
true, writable: true });
  await act(async () => {
    viewport.dispatchEvent(new Event('scroll', { bubbles: true }));
    // flush the rAF the handler queued
    await new Promise(r => requestAnimationFrame(() => r(null)));
  });
};

describe('VirtualizedWindow', () => {
  it('renders ONLY the visible slice plus overscan, not 100,000 rows', () =>
{
    renderFeed();
    const rendered = screen.getAllByRole('row');
    expect(rendered.length).toBeLessThan(40); // ~20 visible + 4
overscan

```

```

    expect(rendered.length).toBeGreaterThan(15);
  });

  it('keeps the DOM node count CONSTANT across a 50,000px scroll jump', async
  () => {
    renderFeed();
    const before = screen.getAllByRole('row').length;
    await scrollTo(50_000);
    const after = screen.getAllByRole('row').length;
    expect(after).toBe(before); // constant cost at any
depth
  });

  it('renders the CORRECT rows after scrolling, not just the right count',
  async () => {
    renderFeed();
    await scrollTo(36 * 1_000);
    expect(screen.queryByText('agent-0')).toBeInTheDocument(); // index 1000
-> 1000%50
    const firstRow = screen.getAllByRole('row')[0];
    expect(Number(firstRow.getAttribute('aria-
rowindex'))).toBeGreaterThan(900);
  });

  it('reports the TRUE total in aria-rowcount, not the rendered slice', () =>
  {
    renderFeed();
    expect(screen.getByRole('grid')).toHaveAttribute('aria-rowcount',
'100000');
  });

  it('gives every row an ABSOLUTE aria-rowindex', async () => {
    renderFeed();
    await scrollTo(36 * 500);
    const indices = screen
      .getAllByRole('row')
      .map(r => Number(r.getAttribute('aria-rowindex')));
    expect(Math.min(...indices)).toBeGreaterThan(400);
    expect(indices).toEqual([...indices].sort((a, b) => a - b)); //
contiguous, ordered
  });

  it('announces the window position politely', async () => {
    renderFeed();
    const status = screen.getByRole('status');
    expect(status).toHaveAttribute('aria-live', 'polite');
  });

```

```

    expect(status).toHaveTextContent(/of 100000/);
  });

  it('coalesces a burst of scroll events into ONE commit per frame', async () => {
    const rafSpy = jest.spyOn(window, 'requestAnimationFrame');
    renderFeed();
    const viewport = screen.getByRole('grid');
    rafSpy.mockClear();
    await act(async () => {
      for (let i = 0; i < 50; i++) {
        viewport.dispatchEvent(new Event('scroll', { bubbles: true }));
      }
    });
    expect(rafSpy).toHaveBeenCalledTimes(1);      // 50 events, one frame
    queued
    rafSpy.mockRestore();
  });

  it('supports keyboard navigation: Down, Up, PageDown, PageUp, Home, End',
  async () => {
    const user = userEvent.setup();
    renderFeed();
    const viewport = screen.getByRole('grid');
    const scrollSpy = jest.fn();
    viewport.scrollTo = scrollSpy as unknown as typeof viewport.scrollTo;

    viewport.focus();
    expect(viewport).toHaveFocus();

    await user.keyboard('{ArrowDown}');
    expect(scrollSpy).toHaveBeenLastCalledWith({ top: 36, behavior: 'auto'
  });

    await user.keyboard('{PageDown}');
    expect(scrollSpy).toHaveBeenLastCalledWith({ top: 36 * 21, behavior:
'auto' });

    await user.keyboard('{End}');
    expect(scrollSpy).toHaveBeenLastCalledWith({ top: 36 * 99_999, behavior:
'auto' });

    await user.keyboard('{Home}');
    expect(scrollSpy).toHaveBeenLastCalledWith({ top: 0, behavior: 'auto' });
  });

```

```

it('does not scroll past the bounds', async () => {
  const user = userEvent.setup();
  renderFeed();
  const viewport = screen.getByRole('grid');
  const scrollSpy = jest.fn();
  viewport.scrollTo = scrollSpy as unknown as typeof viewport.scrollTo;
  viewport.focus();
  await user.keyboard('{ArrowUp}');
  expect(scrollSpy).toHaveBeenLastCalledWith({ top: 0, behavior: 'auto' });
// clamped
});

it('cancels its pending animation frame on unmount', () => {
  const cancelSpy = jest.spyOn(window, 'cancelAnimationFrame');
  const { unmount } = renderFeed();
  const viewport = screen.getByRole('grid');
  viewport.dispatchEvent(new Event('scroll', { bubbles: true }));
  unmount();
  expect(cancelSpy).toHaveBeenCalled(); // no leaked frame, no leaked
closure
  cancelSpy.mockRestore();
});
});

// ===== 5. Profiler commit timing =====

describe('commit budget', () => {
  it('keeps the initial commit within the frame budget', () => {
    const onRender = jest.fn() as
jest.MockedFunction<React.ProfilerOnRenderCallback>;
    render(
      <Profiler id="feed" onRender={onRender}>
        <VirtualizedTelemetryFeed rows={rows} threshold={150} onSelect=
{jest.fn()} />
      </Profiler>,
    );
    expect(onRender).toHaveBeenCalled();
    const [, phase, actualDuration] = onRender.mock.calls[0];
    expect(phase).toBe('mount');
    // A guard against regressions, not a hard performance claim in JSDOM
    expect(actualDuration).toBeLessThan(100);
  });

  it('does NOT commit when a scroll leaves the window indices unchanged',
  async () => {
    const onRender = jest.fn() as

```

```

jest.MockedFunction<React.ProfilerOnRenderCallback>;
  render(
    <Profiler id="feed" onRender={onRender}>
      <VirtualizedTelemetryFeed rows={rows} threshold={150} onSelect=
{jest.fn()} />
    </Profiler>,
  );
  onRender.mockClear();
  await scrollTo(3);          // 3px: still inside the same row window
  expect(onRender).not.toHaveBeenCalled(); // the bail-out works
});
});

```

The four assertions this lab exists for. Constant DOM node count across a 50,000-pixel jump proves the windowing is real. One `requestAnimationFrame` per fifty scroll events proves the coalescing works. `aria-rowcount="100000"` proves the accessibility contract is honest rather than decorative. And no commit on a 3-pixel scroll proves the bail-out — the difference between a virtualized list that is fast and one that merely renders fewer nodes while still committing at 100 Hz.

Part 5 — Active recall flashcards

Review on a 1 / 3 / 7 / 21-day schedule. **Tier A** covers optimization invariants and Profiler diagnostics. **Tier B** is virtualization and Suspense. **Tier C** is Web Vitals and bundler chunking.

#	Tier	Prompt	Answer
1	A	What does <code>React.memo</code> actually do to the Fiber walk?	It gates one node. If props are shallow-equal by <code>Object.is</code> , React skips the component function and reuses the entire subtree — no render, no diff, no commit.
2	A	Why does <code>React.memo</code> often do nothing?	The parent passes a fresh object, array or arrow function. One unstable reference reopens the gate, and the failure is completely silent.
3	A	What is the first rule of performance work?	No optimization ships without a profile that justifies it. Measure in a production build on throttled hardware, then re-measure and record the delta.

#	Tier	Prompt	Answer
4	A	The Profiler shows near-zero render time but the interaction feels slow. What is wrong?	Nothing in React. The bottleneck is the network. Memoizing components would waste a sprint.
5	A	What does "why did this component render?" give you?	The Profiler names the changed prop or hook index. That attribution <i>is</i> the diagnosis; everything after it is a lookup.
6	A	Why can't <code>React.memo</code> fix a context cascade?	Context bypasses props entirely. Every consumer re-renders on value identity change regardless of memoization. Split the context instead.
7	A	Where do you find a memory leak, and what do you look for?	Two heap snapshots around a mount/unmount cycle, filtered for Detached nodes. The retainer path names the listener, timer or closure holding them.
8	B	Why must an error boundary sit outside the Suspense boundary?	Inside, a chunk-load rejection unmounts the Suspense boundary along with its fallback, so the error escapes anyway and the fallback never renders.
9	B	What breaks if <code>lazy()</code> is called inside a component body?	A new component type every render, so React destroys and remounts the subtree each time — state lost, data refetched, fallback flashing.
10	B	Why is a <code>sessionStorage</code> guard mandatory in a chunk-retry wrapper?	Without it a genuinely missing chunk produces an infinite reload loop. The guard turns the second failure into an honest error page.
11	B	Suspense fallback or <code>useTransition</code> ?	Fallback for a first load where there is nothing to keep. Transition for a subsequent load where the current UI should stay visible and interactive.

#	Tier	Prompt	Answer
12	B	What is the core windowing formula?	<code>startIndex = floor(scrollTop / rowHeight)</code> , <code>visibleCount = ceil(viewportHeight / rowHeight)</code> , then apply overscan and clamp to <code>[0, total]</code> .
13	B	Why <code>translate3d</code> rather than <code>top</code> for the row offset?	It is composited on the GPU rather than triggering layout, so scrolling never reflows (MOD-03.1's transform rule).
14	B	What does overscan buy, and what does too much cost?	A buffer against blank gaps during fast scrolling. Too much renders rows nobody sees. Measure it; 3-5 is the usual range.
15	B	Why must a virtualized grid report <code>aria-rowcount</code> as the true total?	Reporting the rendered slice tells a screen-reader user the dataset has 40 rows. Incomplete ARIA is worse than none.
16	B	What makes a virtualized scroll handler fast?	rAF coalescing, one <code>scrollTop</code> read, purely arithmetic indices, and a bail-out when the indices have not changed.
17	C	Name the three Core Web Vitals targets.	LCP < 2.5 s, INP < 200 ms, CLS < 0.1 — at the 75th percentile of real users, not on a developer machine.
18	C	Why is INP harder than the FID it replaced?	FID measured only the delay before a handler started. INP measures the whole path to the next paint, across the entire session.
19	C	What causes CLS in a React SPA?	Fallbacks sized differently from their content, media without explicit dimensions, and content inserted above what is already on screen.
20	C	What should never be code-split?	Anything under ~30 kB, anything on the critical render path, and anything needed immediately after mount. The round trip costs more than the bytes.

#	Tier	Prompt	Answer
21	C	Why does a lazy import need <code>{ default: ... }</code> ?	<code>React.lazy</code> expects a module whose default export is the component. A named export needs an adapter that maps it, with the types preserved.
22	C	Why prefetch on <code>onFocus</code> as well as <code>onMouseEnter</code> ?	Keyboard users reach a link by focus. Hover-only prefetching gives them a measurably slower application than mouse users.

Part 6 — Enterprise production readiness checklist

No PR touching splitting, Suspense or performance merges until every applicable line is verified. Reviewers cite the line in the review comment.

Dynamic imports and chunk-load resilience

- ☐ Every route is lazily loaded; the admin surface is never in the bundle a read-only user downloads
- ☐ Every `lazy()` call is at **module scope**, never inside a component body
- ☐ Every dynamic import goes through the chunk-retry wrapper — exponential backoff, jitter, and a one-shot reload guard
- ☐ A genuine module error is not retried; only recognized chunk-load failures are
- ☐ Every lazy boundary has an **error boundary outside it** rendering a version-mismatch fallback with a reload action
- ☐ Named exports are adapted with a typed helper, never by abandoning the split
- ☐ Nothing under ~30 kB, on the critical path, or needed immediately after mount is split
- ☐ Prefetch on both `onMouseEnter` and `onFocus`
- ☐ Chunk-retry behaviour verified against a simulated 404, not only in theory
- ☐ Long-session behaviour tested: open a tab, deploy, navigate — the recovery path must work

Suspense boundaries and CLS prevention

- ☐ Every fallback matches the final content's dimensions; no unsized spinners in a live layout
- ☐ Skeletons carry `role="status"` and `aria-busy="true"` with a meaningful label
- ☐ Skeleton animation is disabled under `prefers-reduced-motion: reduce`

- ☐ Boundaries are grouped by what should appear together; a page does not assemble in six visible stages
- ☐ Subsequent loads use `useTransition` so the current UI stays visible and interactive
- ☐ `isPending` is surfaced with `aria-busy` on the trigger, not only as a visual label change
- ☐ Images and embeds carry explicit `width` and `height`
- ☐ Nothing is inserted above existing content after first paint
- ☐ CLS measured on the actual route, not assumed

Profiler-verified re-render budgets

- ☐ A before-and-after Profiler measurement is attached to any PR claiming a performance improvement
- ☐ Profiling performed on a production build with CPU throttling, never in development
- ☐ Every `React.memo` is paired with stable props — `useCallback` handlers, hoisted or memoized objects, primitives where possible
- ☐ No `React.memo`, `useMemo` or `useCallback` added without a named consumer or a measurement
- ☐ No custom `areEqual` comparator used to compensate for an unstable prop; stabilize at the source
- ☐ Handlers take an id argument rather than closing over a row, so one stable function serves every row
- ☐ Context values are memoized and split by volatility; no fast-changing data in a widely consumed context
- ☐ High-frequency data (WebSocket, polling) is routed through per-subscriber selection, not a context value
- ☐ Commit duration for the main interactive surface stays within the frame budget under realistic data volume
- ☐ A `<Profiler>` reports slow commits to telemetry in production for the critical surfaces

Virtualization hygiene and memory stability

- ☐ Any list that can exceed ~200 rows is virtualized
- ☐ Window indices are computed arithmetically; no `getBoundingClientRect` or `offsetHeight` inside a scroll handler
- ☐ Scroll handlers are rAF-coalesced and registered `{ passive: true }`
- ☐ The window state bails out when indices are unchanged, so a small scroll costs no commit

- ☐ Row offset uses `transform: translate3d`, never `top` or `margin`
- ☐ Pending animation frames are cancelled on unmount
- ☐ Every listener, timer, subscription and event-bus registration has a teardown
- ☐ Heap verified: mount and unmount the feature ten times, force GC, and confirm no detached nodes accumulate
- ☐ Retained heap returns to baseline after navigating away
- ☐ Overscan value chosen by measurement, not by default

Core Web Vitals compliance

- ☐ LCP < 2.5 s, INP < 200 ms, CLS < 0.1 at the 75th percentile of **real users**
- ☐ Real User Monitoring via `web-vitals` reports by route, device class and region
- ☐ No long task over 200 ms on any primary interaction path
- ☐ Bundle budget enforced in CI; a regression fails the build rather than appearing in a report nobody reads (APP-01)
- ☐ Tree-shaking verified for large dependencies; no locale or icon set imported wholesale
- ☐ Lighthouse CI runs on the critical routes per PR
- ☐ Performance regressions are traceable to a commit because the budget is enforced per merge

RTL performance and viewport testing

- ☐ Windowing math tested as a pure function: boundaries, clamping, empty list, deep scroll
 - ☐ DOM node count asserted constant across a large scroll jump
 - ☐ Correct rows asserted after scrolling, not only the correct count
 - ☐ rAF coalescing asserted: N scroll events produce one frame
 - ☐ Bail-out asserted: a sub-row scroll produces no commit
 - ☐ `aria-rowcount`, absolute `aria-rowindex` and the live region asserted
 - ☐ Keyboard navigation covered for Arrow keys, PageUp, PageDown, Home and End, including clamping at both bounds
 - ☐ Frame cancellation on unmount asserted
 - ☐ Chunk-retry recovery, reload-once behaviour and no-retry-on-real-errors all covered
 - ☐ Suspense fallback asserted for `aria-busy` and dimensional match
 - ☐ Profiler commit timing asserted as a regression guard
 - ☐ Queries use `getByRole` / `getByLabelText`; no test IDs for user-facing elements
-

Exit criteria for MOD-07. LAB-07 passes the full suite, including the constant-node-count test across a 50,000 px scroll, the single-rAF coalescing test, the reload-once chunk guard, and the no-commit-on-small-scroll bail-out; everything compiles under `strict` with zero `any` ; all seven Tier A cards answer cold; and the learner can, unaided, take a slow enterprise screen, profile it, name the root cause from the flamegraph, apply the narrowest fix, and demonstrate the delta.

End of the core curriculum. MOD-01 through MOD-07 are complete. **APP-01 — Enterprise Build Tooling and CI/CD Pipelines** is compiled next, followed by the **SDLC and GenAI Governance** chapter.